

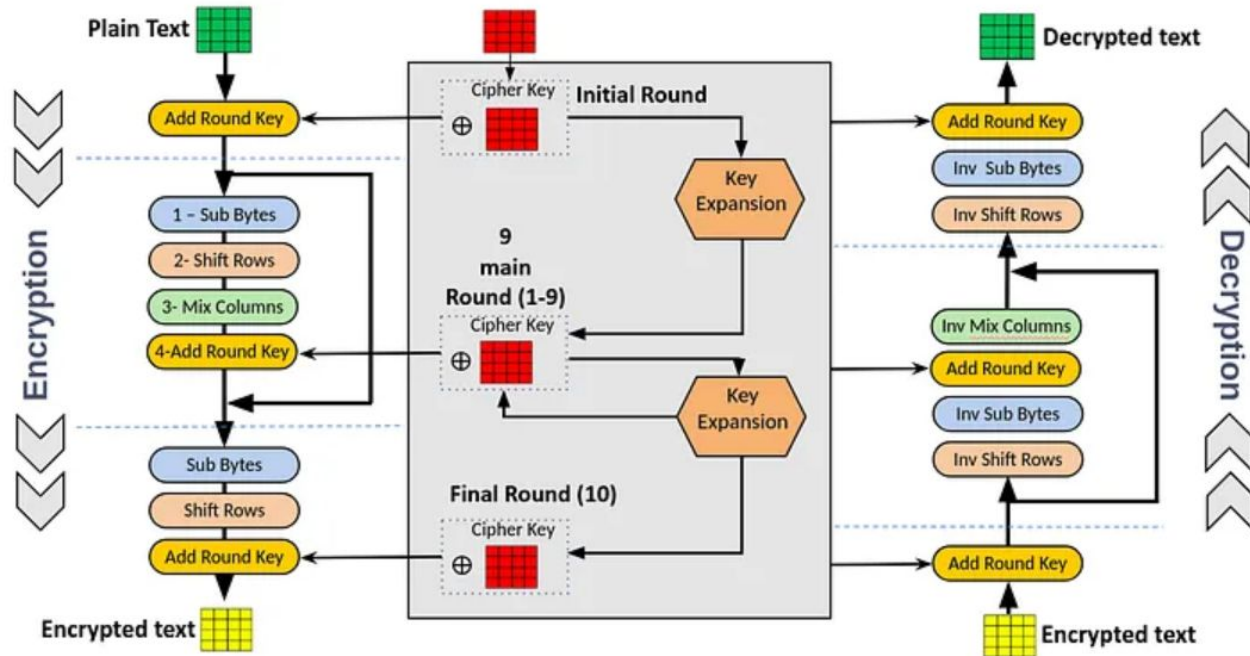
Atividade AES

Nome: José Henrique Ioki Yamaoki

Introdução ao AES

- O AES (Advance Encryption Standard) é um algoritmo de criptografia de chave simétrica. Amplamente usado nos dias atuais, usado em aplicações de troca de mensagens, segurança de redes wifi (WPA2 e WPA3), gerenciadores de senha, armazenamento em nuvem e entre outros.
- O AES precisa de 2 componentes de entrada, uma mensagem e um chave. A mensagem precisa ser quebrada em blocos de 128 bits (16 bytes), esses blocos são organizados no formato de uma matriz 4x4. A chave vai variar de tamanho conforme a variação escolhida do AES, são elas AES-128, AES-192 e AES-256.
- Os blocos de mensagem e a chave são submetidas a uma série de operações para criptografar os blocos e então retornar os blocos criptografados.
- Para descriptografar, deve ser fornecido os blocos de mensagem encriptados e a mesma chave, então realizar as operações inversas àquelas que foram usadas para encriptar.

Introdução ao AES - Diagrama



Desenvolvimento do AES-128

- Foi desenvolvido um módulo `aes_base.py`, que contém os códigos que são comuns a todos os AES: `generate_msg_block`, `rot_word`, `sub_word`, `xor_words`, `add_round_key`, `sub_bytes`, `shift_rows`, `aes_round`, `inv_sub_bytes`, `inv_shift_rows`, `inv_mix_columns`.
- O módulo AES-128 precisa implementar as funções `fix_key_size`, `expand_key`, `aes_encrypt`, `aes_decrypt` e definir `KEY_SIZE`, nesse caso essa constante é definida com valor 16, tamanho em bytes.
- A função `fix_key_size` fixa o tamanho da chave com o tamanho `KEY_SIZE`, truncando quando a chave passada for maior que `KEY_SIZE`, ou completando com 0s.
- No AES-128 a função `expand key` precisa gerar 11 chaves de rodadas e função `encrypt` precisa rodar 10 rounds.

Resultados

- Mensagem: teste
- Chave: chave
- Mensagem cifrada: 7c7663d6a0241fa119546951f219d5b0
- Mensagem alterada (bit 0): testd
- Mensagem alterada cifrada: 26a9a82099edf89116210873315ab166

Desenvolvimento do AES-192

- No AES-192, a constante `KEY_SIZE` é definida como 24, correspondente ao tamanho da chave em bytes. A função `fix_key_size` fixa o tamanho da chave em `KEY_SIZE`, truncando quando a chave passada for maior que `KEY_SIZE`, ou completando com 0s quando for menor.
- No AES-192, a função `expand_key` precisa gerar 13 chaves de rodadas, sendo uma chave inicial e mais 12 chaves correspondentes às rodadas. A função `aes_encrypt` precisa executar 12 rounds de criptografia.

Resultados

- Mensagem: teste
- Chave: chave
- Mensagem cifrada: 424d45a440bbd470a9a8e9edb740d40b
- Chave alterada (bit 0): chavd
- Mensagem cifrada com chave alterada: ab7998dc33e87776ca2542ceb3d741a4

Desenvolvimento do AES-256

- No AES-256, a constante `KEY_SIZE` é definida como 32, correspondente ao tamanho da chave em bytes. A função `fix_key_size` fixa o tamanho da chave em `KEY_SIZE`, truncando quando a chave passada for maior que `KEY_SIZE`, ou completando com 0s quando for menor.
- No AES-256, a função `expand_key` precisa gerar 15 chaves de rodadas, sendo uma chave inicial e mais 14 chaves correspondentes às rodadas. A função `aes_encrypt` precisa executar 14 rounds de criptografia.

Resultados

- Mensagem: teste
- Chave: chave
- Mensagem cifrada: 974f023982e016d5719aa5641b5b62ad
- Mensagem alterada (bit 0): testd
- Mensagem alterada cifrada: deab18e3acafef86d944a2861cc64e24

Efeito Avalanche

- **Pequena alteração na mensagem:** a alteração de apenas um bit da mensagem original, resultando de teste para testd, provocou uma mudança significativa no texto cifrado.
- **Grande alteração no texto cifrado:** comparando **7c7663d6a0241fa119546951f219d5b0** com **26a9a82099edf89116210873315ab166**, observa-se que diversos bytes foram modificados, apesar da pequena alteração realizada na mensagem de entrada.
- **Efeito avalanche:** esse comportamento demonstra o efeito avalanche, uma propriedade desejável em algoritmos criptográficos na qual uma pequena mudança na entrada provoca uma grande e aparentemente imprevisível alteração na saída, dificultando a identificação de relações entre mensagens e seus respectivos textos cifrados.

Comparação entre os algoritmos

- AES-128: utiliza chaves menores e menos rodadas, apresentando o menor custo computacional entre as três versões.
- AES-192: possui uma chave maior e mais rodadas que o AES-128, resultando em um aumento moderado no tempo de execução.
- AES-256: utiliza a maior chave e o maior número de rodadas, apresentando o maior custo computacional, especialmente durante a expansão da chave e a cifragem.

Comparação entre os algoritmos

- AES-128: oferece 128 bits de segurança de chave, sendo suficiente para aplicações que não exigem um nível de segurança superior.
- AES-192: aumenta o espaço de chaves para 192 bits, proporcionando maior resistência a ataques de força bruta em comparação ao AES-128.
- AES-256: possui 256 bits de segurança de chave e o maior número de rodadas, oferecendo a maior margem de segurança entre as três versões, ao custo de maior processamento.

Desafios técnicos

- Implementação da expansão da chave: tive dificuldade para entender e implementar corretamente o Key Schedule, principalmente porque cada versão do AES trabalha com um tamanho de chave diferente e exige uma quantidade diferente de palavras e rodadas.
- Implementação das rodadas do AES: tive dificuldade em controlar corretamente a sequência de operações como SubBytes, ShiftRows, MixColumns e AddRoundKey, incluindo suas versões inversas durante a descryptografia. Além disso, precisei considerar a diferença na quantidade de rodadas entre o AES-128, AES-192 e AES-256.

Conclusão

- **Validação prática do efeito avalanche:** A alteração de apenas um bit na mensagem original provoca uma mudança ampla e imprevisível em diversos bytes do texto cifrado, confirmando que a implementação atinge essa propriedade essencial da criptografia.
- **Equilíbrio entre segurança e desempenho:** A comparação entre as versões evidencia que o AES-128 apresenta o menor custo computacional, o AES-192 amplia o espaço de chaves contra ataques de força bruta, e o AES-256 oferece a maior margem de segurança ao custo de mais rodadas e processamento.
- **Arquitetura modular e reutilização de código:** A centralização das operações fundamentais no módulo ``aes_base.py`` permitiu reaproveitar a estrutura do algoritmo para suportar as diferentes variações de chave (128, 192 e 256 bits) de forma organizada, garantindo também a simetria no processo de cifragem e descifragem